

05_Spark_Practica_Taxis_NYC_executed

March 11, 2026

1 Práctica 2: Taxis de NYC - Fechas y Ventanas (Arquitectura Medallón)

Dataset: NYC Taxi Trip Duration **Tecnología:** Apache Spark (Modo Local)

El dataset de Taxis de Nueva York es un clásico. Descargaremos una versión manejable de la competición de Kaggle para probar el manejo de **Timestamps** y aplicar **Window Functions** avanzadas de Spark para crear cuadros de mando (Oro).

Recuerda: Si recibes Error 403 Forbidden al descargar, revisa tus credenciales temporales de `kaggle.json`.

```
[1]: from pyspark.sql import SparkSession
from pyspark.sql.functions import col, hour, dayofweek, max, rank
from pyspark.sql.window import Window

spark = (SparkSession.builder
        .appName("Practica_Taxis_NYC_Medallion")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "16") # prueba 8, 16, 32
        .getOrCreate()
        )

# spark = (
#     SparkSession.builder
#     .appName("Intro_Spark")
#     .master("local[*]")
#     .config("spark.sql.shuffle.partitions", "16") # prueba 8, 16, 32
#     .getOrCreate()
# )
# Ejemplo: muestra la versión de PySpark
try:
    print(f"Versión de PySpark {spark.version}.")
except NameError:
    print("SparkSession no inicializada; ejecuta la celda de creación de spark.
    ↪")
```

Versión de PySpark 3.5.0.

1.0.1 Bronze: Ingesta

```
[2]: !pip install -q opendatasets
import opendatasets as od

dataset_url = "https://www.kaggle.com/datasets/yasserh/nyc-taxi-trip-duration"
od.download(dataset_url)

# Ingestamos en Crudo
!hdfs dfs -mkdir -p /data/bronze/taxi/
!hdfs dfs -put -f nyc-taxi-trip-duration/*.csv /data/bronze/taxi/
```

WARNING: The directory '/home/jovyan/.cache/pip' or its parent directory is not owned or is not writable by the current user. The cache has been disabled. Check the permissions and owner of that directory. If executing pip with sudo, you should use sudo's -H flag.

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead:

<https://pip.pypa.io/warnings/venv>

Please provide your Kaggle credentials to download this dataset. Learn more:

<http://bit.ly/kaggle-creds>

Your Kaggle username:

```
-----
StdinNotImplementedError                                Traceback (most recent call last)
```

```
Cell In[2], line 5
```

```
      2 import opendatasets as od
      4 dataset_url = "https://www.kaggle.com/datasets/yasserh/
↳ nyc-taxi-trip-duration"
----> 5 od.download(dataset_url)
      7 # Ingestamos en Crudo
      8 get_ipython().system('hdfs dfs -mkdir -p /data/bronze/taxi/')
```

```
File /opt/conda/lib/python3.11/site-packages/opendatasets/__init__.py:13, in
```

```
↳ download(dataset_id_or_url, data_dir, force, dry_run, **kwargs)
```

```
      10 def download(dataset_id_or_url, data_dir='.', force=False,
↳ dry_run=False, **kwargs):
```

```
      11     # Check for a Kaggle dataset URL
      12     if is_kaggle_url(dataset_id_or_url):
```

```
----> 13         return
```

```
↳ download_kaggle_dataset(dataset_id_or_url, data_dir=data_dir, force=force, dry_run=dry_run,
```

```
      15     # Check for Google Drive URL
```

```
      16     if is_google_drive_url(dataset_id_or_url):
```

```

File /opt/conda/lib/python3.11/site-packages/opendatasets/utils/kaggle_api.py:
  46, in download_kaggle_dataset(dataset_url, data_dir, force, dry_run)
    44 if not read_kaggle_creds():
    45     print("Please provide your Kaggle credentials to download this_
↳dataset. Learn more: http://bit.ly/kaggle-creds")
---> 46     os.environ['KAGGLE_USERNAME'] = click.prompt("Your Kaggle username"
    47     os.environ['KAGGLE_KEY'] = _get_kaggle_key()
    49 if not dry_run:

File /opt/conda/lib/python3.11/site-packages/click/termui.py:164, in
↳prompt(text, default, hide_input, confirmation_prompt, type, value_proc,
↳prompt_suffix, show_default, err, show_choices)
    162 while True:
    163     while True:
--> 164         value = prompt_func(prompt)
    165         if value:
    166             break

File /opt/conda/lib/python3.11/site-packages/click/termui.py:140, in prompt.
↳<locals>.prompt_func(text)
    137     echo(text.rstrip(" "), nl=False, err=err)
    138     # Echo a space to stdout to work around an issue where
    139     # readline causes backspace to clear the whole line.
--> 140     return f(" ")
    141 except (KeyboardInterrupt, EOFError):
    142     # getpass doesn't print a newline if the user aborts input with ^C.
    143     # Allegedly this behavior is inherited from getpass(3).
    144     # A doc bug has been filed at https://bugs.python.org/issue24711
    145     if hide_input:

File /opt/conda/lib/python3.11/site-packages/ipykernel/kernelbase.py:1201, in
↳Kernel.raw_input(self, prompt)
    1199 if not self._allow_stdin:
    1200     msg = "raw_input was called, but this frontend does not support_
↳input requests."
-> 1201     raise StdinNotImplementedError(msg)
    1202 return self._input_request(
    1203     str(prompt),
    1204     self._parent_ident["shell"],
    1205     self.get_parent("shell"),
    1206     password=False,
    1207 )

StdinNotImplementedError: raw_input was called, but this frontend does not_
↳support input requests.

```

1.0.2 Plata: Validaciones y Transformaciones Temporales

La columna `pickup_datetime` guarda la fecha y hora. En la capa de Plata, los Data Engineers se encargan de **enriquecer** los datos para que negocio lo tenga fácil.

```
[3]: ruta_bronze = "hdfs://namenode:9000/data/bronze/taxi/*.csv"
df_crudo = spark.read.option("header", "true").option("inferSchema", "true").
    ↪ csv(ruta_bronze)
print("Carga de df_crudo")
```

Carga de `df_crudo`

```
[4]: df_crudo.show(1, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+
|id      |vendor_id|pickup_datetime   |dropoff_datetime
|passenger_count|pickup_longitude |pickup_latitude  |dropoff_longitude
|dropoff_latitude |store_and_fwd_flag|trip_duration|
+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+
|id2875421|2        |2016-03-14 17:24:55|2016-03-14 17:32:30|1
|-73.9821548461914|40.76793670654297|-73.96463012695312|40.765602111816406|N
|455      |
+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+
only showing top 1 row
```

```
[5]: df_crudo.show(1, truncate=False, vertical=True)
```

```
-RECORD 0-----
id          | id2875421
vendor_id   | 2
pickup_datetime | 2016-03-14 17:24:55
dropoff_datetime | 2016-03-14 17:32:30
passenger_count | 1
pickup_longitude | -73.9821548461914
pickup_latitude | 40.76793670654297
dropoff_longitude | -73.96463012695312
dropoff_latitude | 40.765602111816406
store_and_fwd_flag | N
trip_duration   | 455
only showing top 1 row
```

```
[6]: df_crudo.printSchema()
```

```
root
 |-- id: string (nullable = true)
 |-- vendor_id: integer (nullable = true)
 |-- pickup_datetime: timestamp (nullable = true)
 |-- dropoff_datetime: timestamp (nullable = true)
 |-- passenger_count: integer (nullable = true)
 |-- pickup_longitude: double (nullable = true)
 |-- pickup_latitude: double (nullable = true)
 |-- dropoff_longitude: double (nullable = true)
 |-- dropoff_latitude: double (nullable = true)
 |-- store_and_fwd_flag: string (nullable = true)
 |-- trip_duration: integer (nullable = true)
```

```
[7]: # 1. Enriquecer los datos derivando dos columnas: el Día de la semana, y la
      ↪ Hora exacta.
df_plata = df_crudo.withColumn("dia_semana", dayofweek(col("pickup_datetime")))\
      ↪\
      .withColumn("hora_recogida", hour(col("pickup_datetime")))
# dayofweek : Extract the day of the week of a given date/timestamp as integer.
      ↪ Ranges from 1 for a Sunday through to 7 for a Saturday
```

```
[8]: df_plata.show(1, truncate=False, vertical=True)
```

```
-RECORD 0-----
id            | id2875421
vendor_id     | 2
pickup_datetime | 2016-03-14 17:24:55
dropoff_datetime | 2016-03-14 17:32:30
passenger_count | 1
pickup_longitude | -73.9821548461914
pickup_latitude | 40.76793670654297
dropoff_longitude | -73.96463012695312
dropoff_latitude | 40.765602111816406
store_and_fwd_flag | N
trip_duration  | 455
dia_semana     | 2
hora_recogida  | 17
only showing top 1 row
```

```
[9]: df_plata.select("id", "vendor_id",\
      ↪ "passenger_count", "dia_semana", "hora_recogida").show(10, truncate=False)
```

```
+-----+-----+-----+-----+-----+
|id      | vendor_id | passenger_count | dia_semana | hora_recogida |
```

id2875421	2	1	2	17
id2377394	1	1	1	0
id3858529	2	1	3	11
id3504673	2	1	4	19
id2181028	2	1	7	13
id0801584	2	6	7	22
id1813257	1	4	6	22
id1324603	2	1	7	7
id1301050	1	1	6	23
id0012891	2	1	5	21

only showing top 10 rows

```
[10]: # 2. Guardar el trabajo limpio
ruta_silver = "hdfs://namenode:9000/data/silver/taxi/"
df_plata.write.mode("overwrite").parquet(ruta_silver)
```

1.0.3 Oro: Analítica Avanzada (Funciones de Ventana)

KPI a la vista: Tu jefe quiere saber todos los detalles técnicos referidos de “cuál fue el viaje de mayor duración PARA CADA UNO de los días de la semana”. Un `groupBy` destruiría el detalle. Usaremos una Ventana.

2 NOTA: QUIERO QUE ME EXPLIQUES PORQUE SE HACE UNA VENTANA Y LO ENTIENDAS Y QUE ME SEPAS EXPLICAR LO QUE HACE EL CODIGO SIGUIENTE

```
[11]: df_silver = spark.read.parquet(ruta_silver)

# 1. Definimos la ventana: El contexto de nuestra búsqueda está particionado/
↳ dentro de "cada día_semana"
ventana_por_dia = Window.partitionBy("dia_semana").orderBy(col("trip_duration").
↳ desc())
```

```
[12]: # 2. Aplicamos la ventana añadiendo un ranking a cada viaje e inyectando la
↳ columna
df_ranking = df_silver.withColumn("ranking_duracion", rank().
↳ over(ventana_por_dia))
df_ranking.select("dia_semana", "trip_duration", "passenger_count",
↳ "pickup_datetime", "ranking_duracion").show(10, truncate=False)
```

dia_semana	trip_duration	passenger_count	pickup_datetime	ranking_duracion

15	86387	1	2016-06-30 16:37:52	1
15	86385	1	2016-06-23 16:01:45	2
15	86378	1	2016-05-12 13:48:19	3
15	86369	6	2016-05-26 14:55:11	4
15	86369	1	2016-02-25 15:17:29	4
15	86361	1	2016-04-14 17:20:48	6
15	86353	6	2016-06-09 20:52:30	7
15	86350	4	2016-06-02 21:47:46	8
15	86348	5	2016-05-26 13:18:44	9
15	86346	2	2016-03-03 22:02:24	10

only showing top 10 rows

```
[13]: # 3. Modelo de Negocio (Oro): Filtramos la medalla de oro de cada día (ranking_
      ↳ = 1)
df_oro = df_ranking.filter(col("ranking_duracion") == 1).select("dia_semana",
      ↳ "trip_duration", "passenger_count", "pickup_datetime")
```

```
[14]: # 4. Publicar en Capa Oro
ruta_gold = "hdfs://namenode:9000/data/gold/taxi_longest_trips/"
df_oro.write.mode("overwrite").parquet(ruta_gold)
df_oro.show()
```

dia_semana	trip_duration	passenger_count	pickup_datetime
1	86369	1	2016-01-17 20:27:56
2	86392	2	2016-02-15 23:18:06
3	2227612	1	2016-01-05 06:14:15
4	86366	4	2016-03-09 02:59:46
5	86387	1	2016-06-30 16:37:52
6	86390	1	2016-05-06 00:00:10
7	3526282	1	2016-02-13 22:46:52

```
[15]: # Cerrar la sesión de Spark al finalizar
try:
    spark.stop()
    print("SparkSession detenida.")
except Exception as e:
    print("No se pudo detener SparkSession:", e)
```

SparkSession detenida.